Apache XMLBeans 1.0

An Innovative Implementation of
the XML Schema Type System

XMLBeans v1 Design Principle

- Enable use of Java variables whose type is ***truly*** an XML Schema type.
  - A compiler that translates *every* schema type into a precisely corresponding Java type.
  - Runtime support for *100%* XML Schema and *100%* XML infoset.
  - A design that is *natural* to use within Java (i.e., early-bound JavaBean classes).

Background: Schema Example

```
<person>
  <name>John Jones</name>
  <birthday>01-05-1970</birthday>
</person>

<xs:schema . . .>
  <xs:element name="person" type="person-info"/>
  <xs:complexType name="person-info">
    <xs:sequence>
      <xs:element name="name" type="xs:string"/>
      <xs:element name="birthday" type="xs:date"/>
    </xs:sequence>
  </xs:complexType>
</xs:schema>
```

Getting the data out: Two ways

```
<person>
  <name>John Jones</name>
  <birthday>01-05-1970</birthday>
</person>
```

#1: With Java type translation:

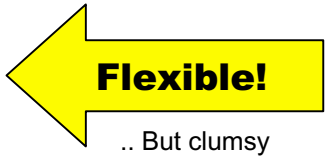
```
String name = doc.getPerson().getName();
Date bd = doc.getPerson().getBirthday();
```

#2: Without Java type translation:

```
NodeList nodes = dom.getChildNodes();
for (int i = 0; i < nodes.length(); i++)
{
    if (nodes.item(i).getNodeName().equals("name"))
        name = nodes.item(i).getNodeValue();
    if (nodes.item(i).getNodeName().equals("birthday"))
        bd = nodes.item(i).getNodeValue();
}
```



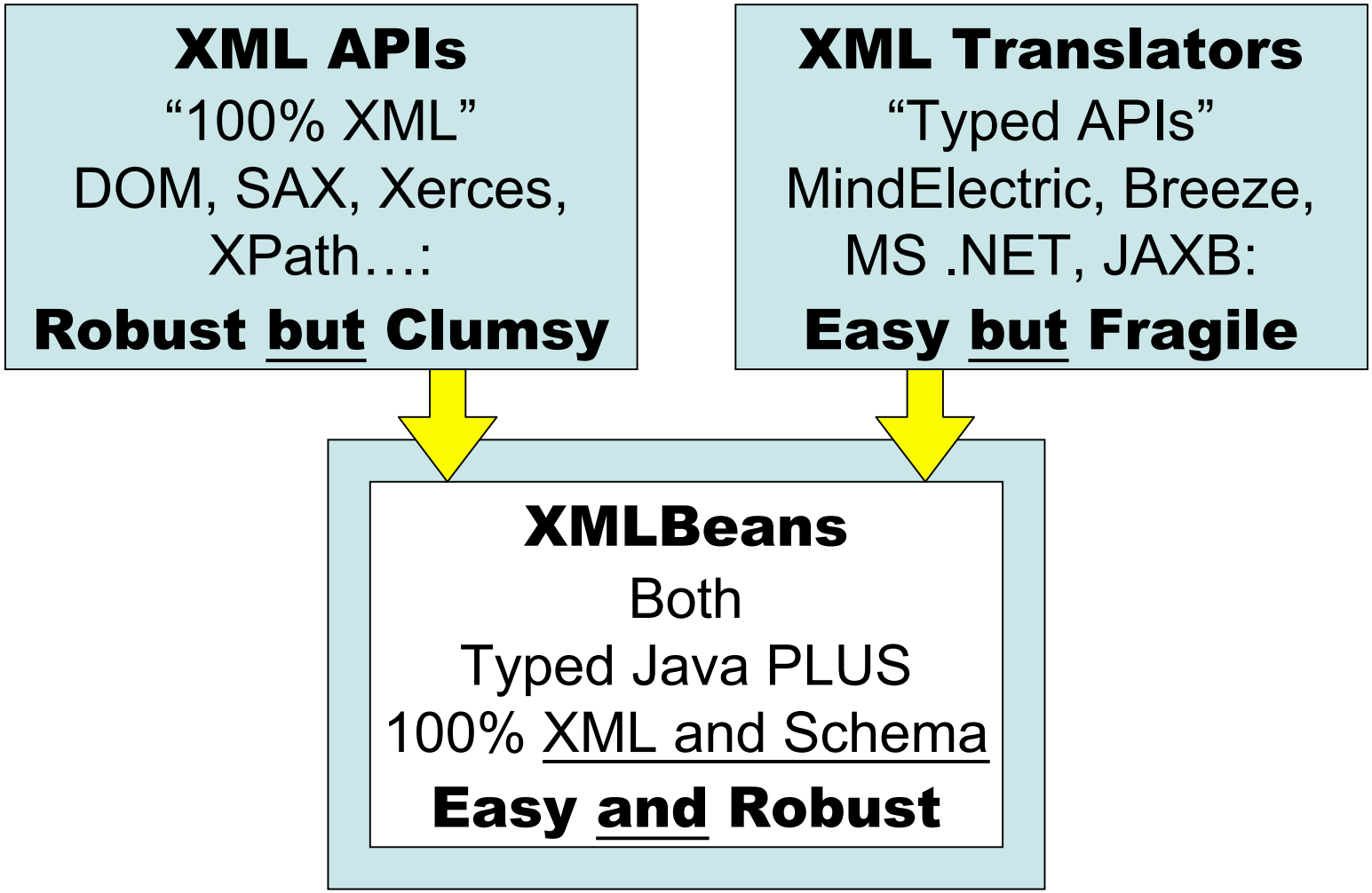
.. But fragile



.. But clumsy

Today it is a dilemma

Why?


XML APIs
“100% XML”
DOM, SAX, Xerces,
XPath…:
Robust but Clumsy
XML Translators
“Typed APIs”
MindElectric, Breeze,
MS .NET, JAXB:
Easy but Fragile
XMLBeans
Both
Typed Java PLUS
100% XML and Schema
Easy and Robust

Versus Existing Solutions

- JAXB, Castor, and many other XML binding solutions already exist. “The area bores the heck out of me.” So why?
- The hole: none actually implement the whole XML Schema data model as Java instance data.
- E.g., variously they give up on major portions of Schema such as type substitution, relationship to full XML infoset, use of type restrictions, etc.
- XML Schema is a programming language. It is simply not enough to implement 50% of it.

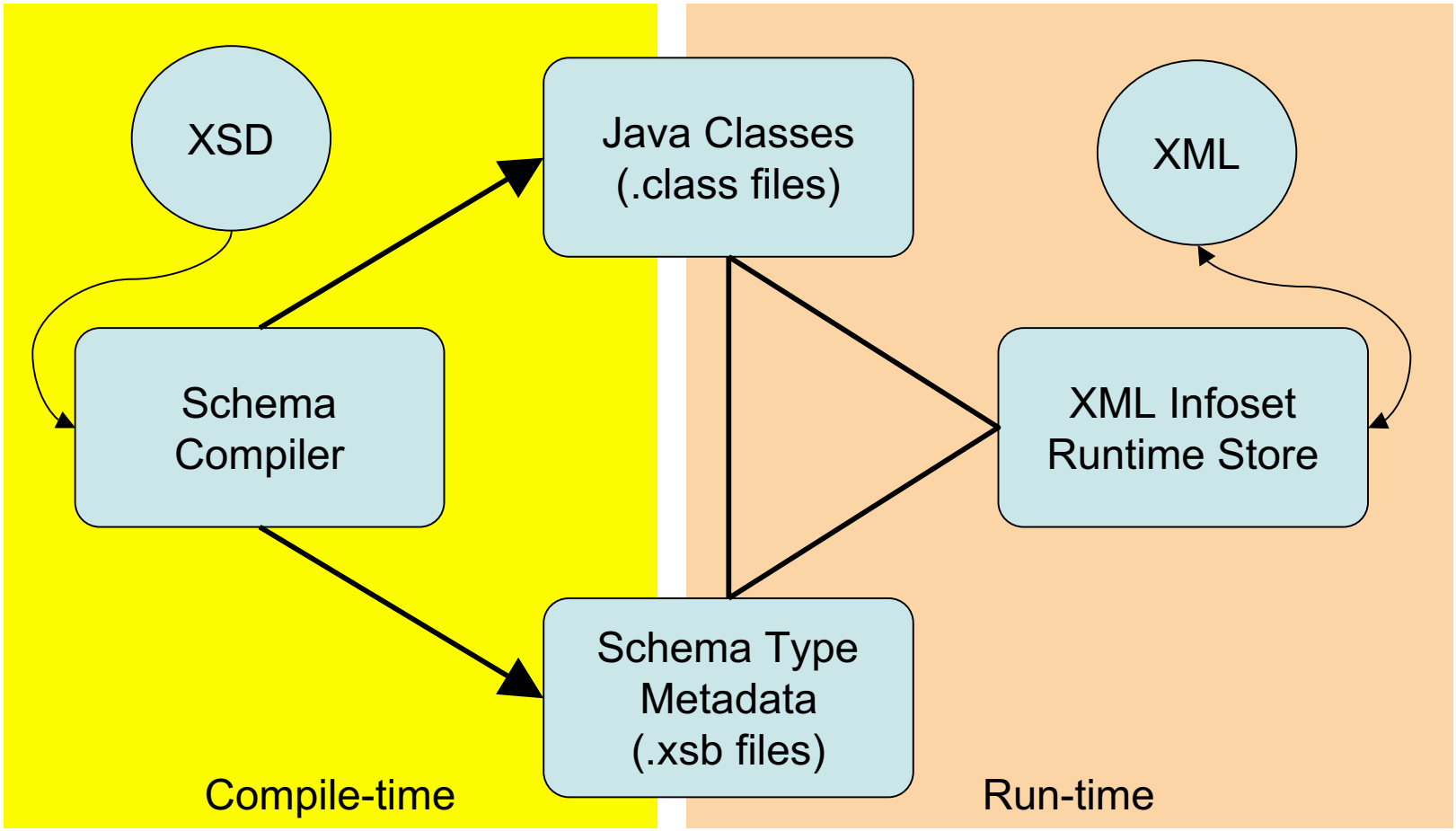
What

- Demo
- Steps to use XMLBeans:
  1. Make XML instances via XmlObject.Factory
  2. Use generic XML-cursor APIs
  3. Compile your schemas
  4. Use type-specific compiled APIs
  5. Validate when needed
  6. Use SchemaType reflection when needed

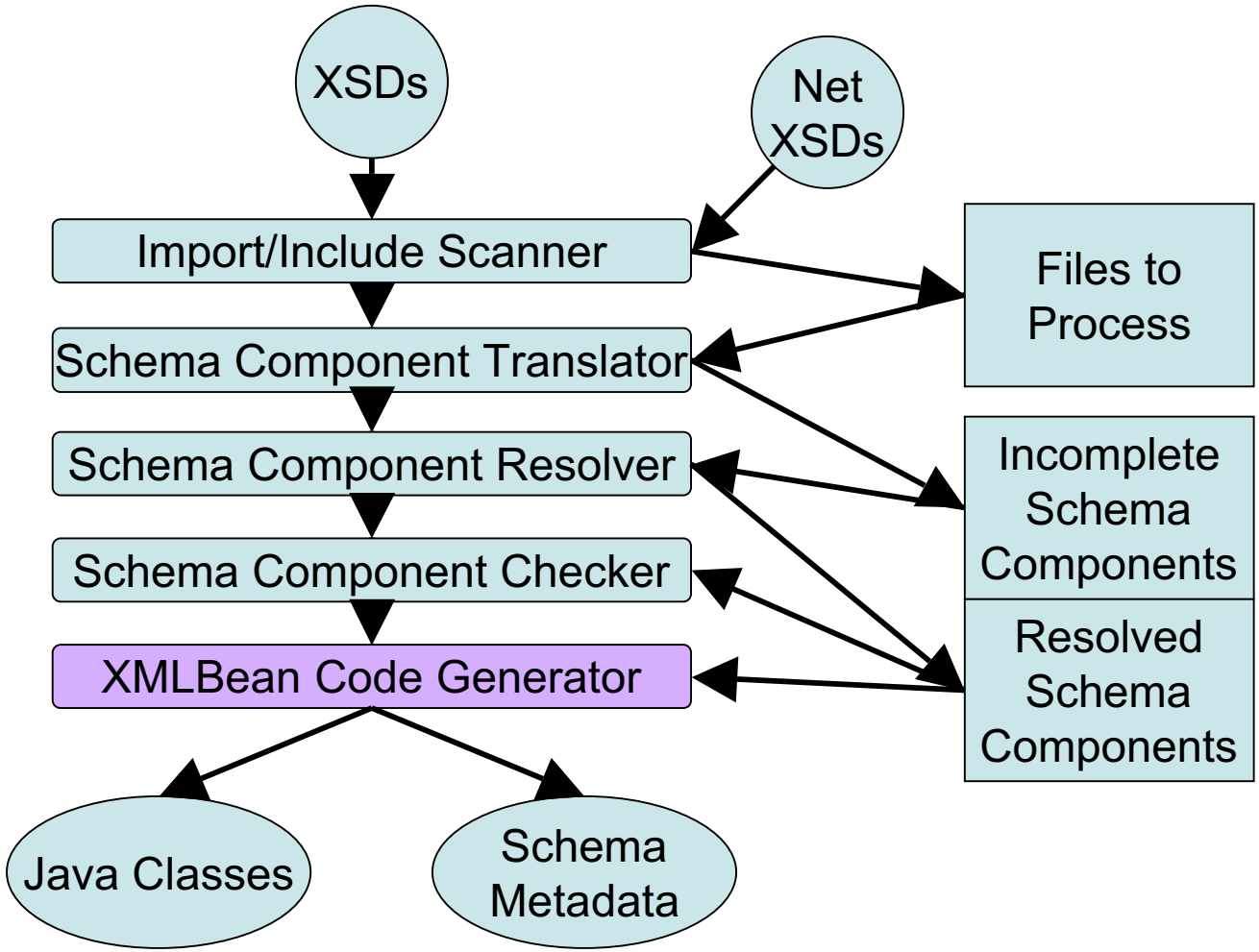
Demo

- Simple usage: load, manipulate, save an XML document in very few lines of code.
- Notice how XML Schema is being used as a programming language for Java.

How


XSD
Java Classes
(.class files)
XML
Schema
Compiler
XML Infoset
Runtime Store
Schema Type
Metadata
(.xsb files)
Compile-time
Run-time

The Compiler



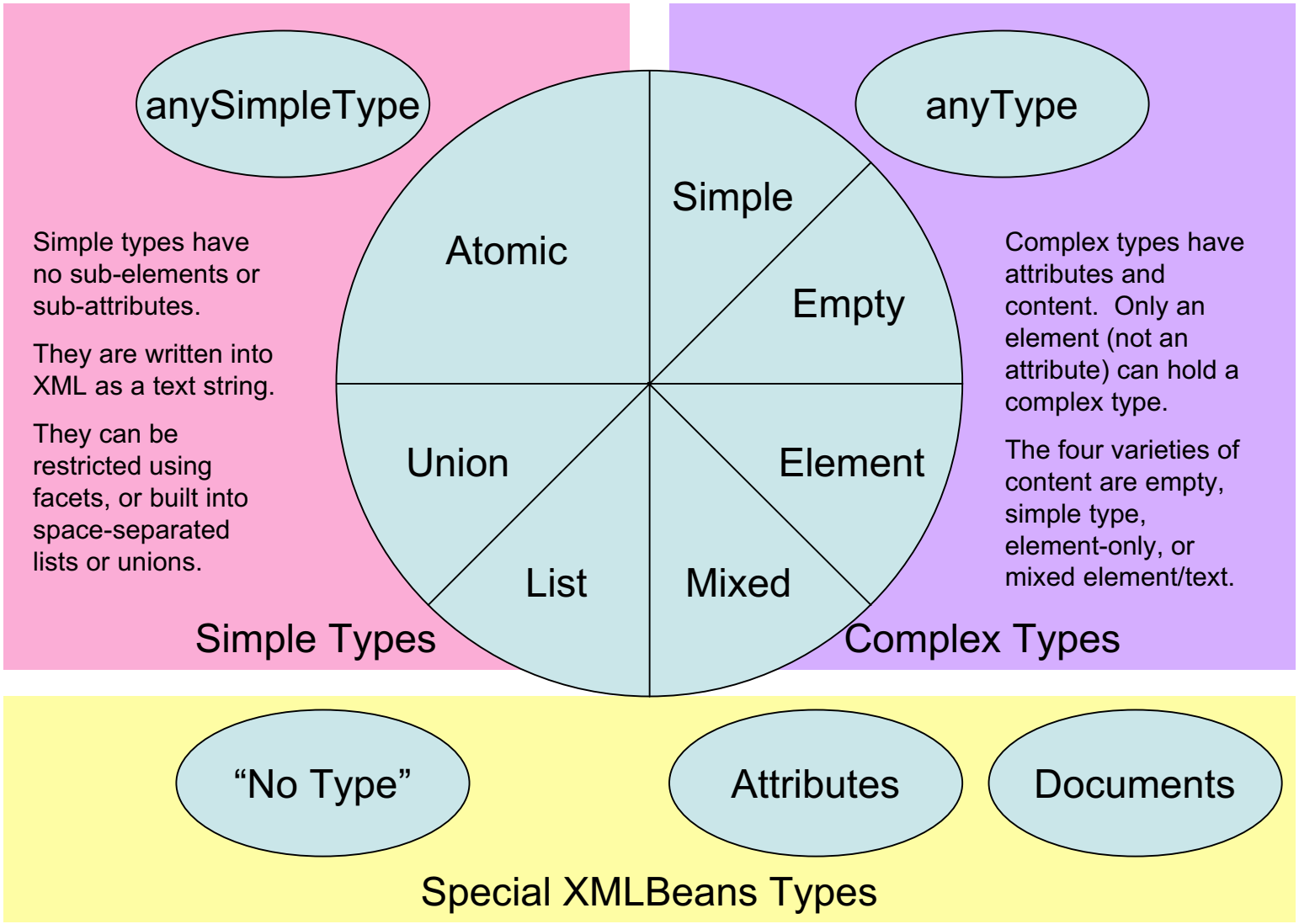
Codegen

- Java and schema types are one-to-one
- Getters must be as easy as basic XPath
- 100% of Schema must be supported
- 100% of XML data must be available
- No compiler configuration necessary

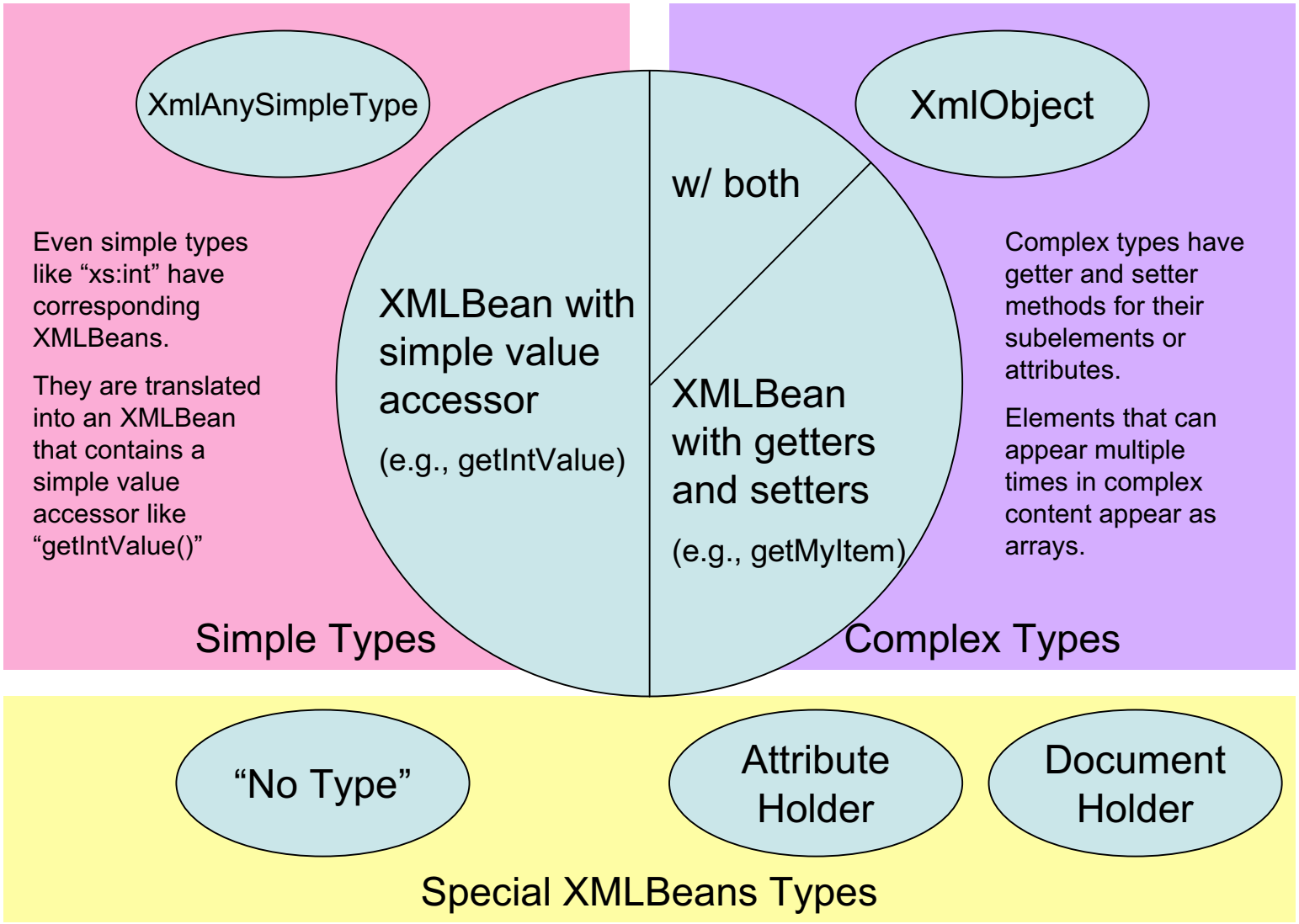
Tricky Codegen Problems

- No **types for documents** in Schema.
- No **user-defined simple types** in Java.
- No **inheritance by restriction** in Java.
- No **complex content models** in Java.
- No **wildcards** in Java.

Taxonomy of Schema Types



Taxonomy of Java Schema Types



Keys about Type Translation

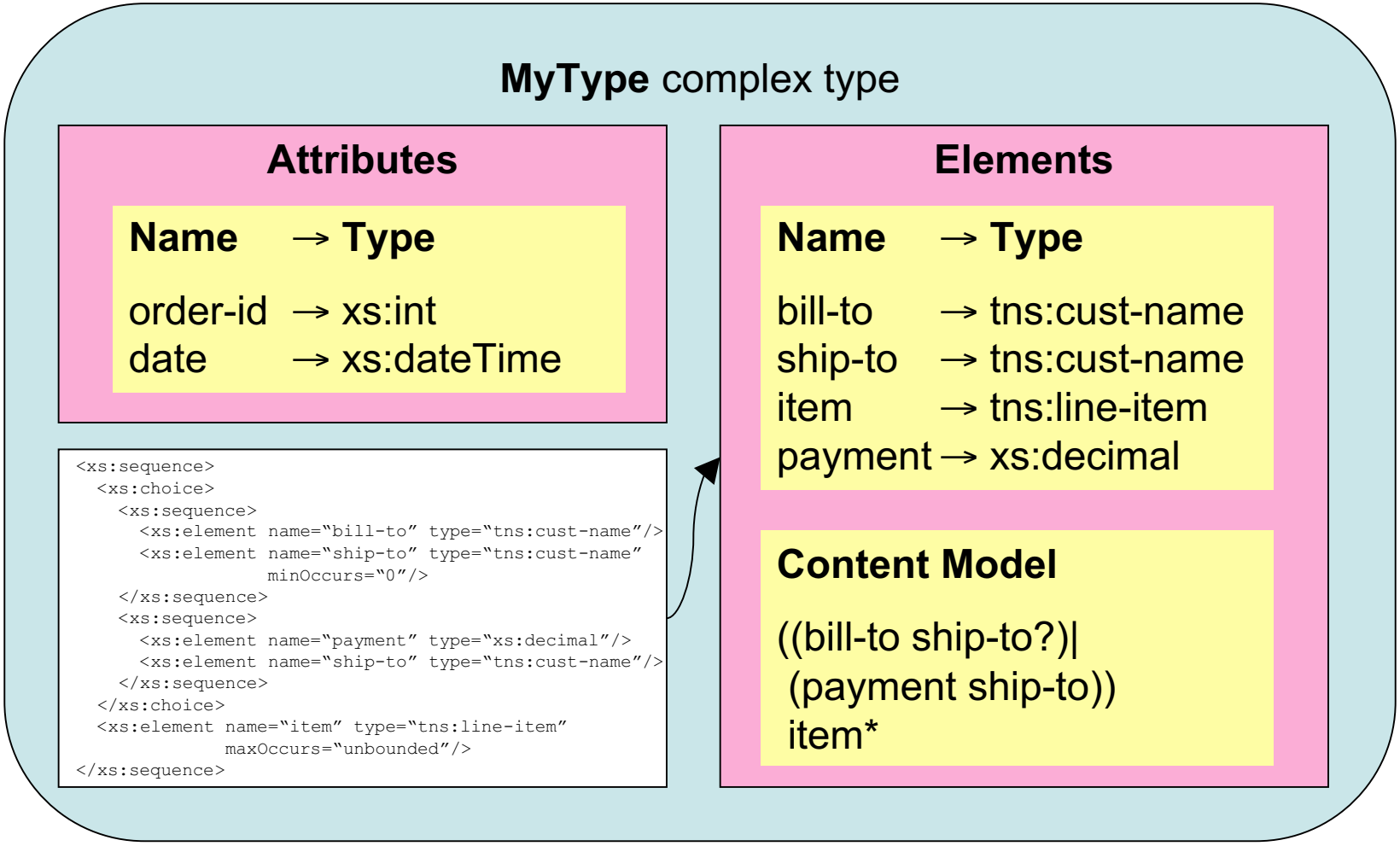
- Type translation is exactly one-to-one. *Even simple XML types* like xs:int correspond to Java classes like XmlInt.
- Restriction is not reflected in signatures, but in behavior (e.g., getItemArray() is always empty).
- This allows inheritance to be exactly one-to-one.
- But then we want:
  `int i = xmlobj.getQuantity();`
  yet `int` cannot be subclassed as in schema.

Getters and XGetters

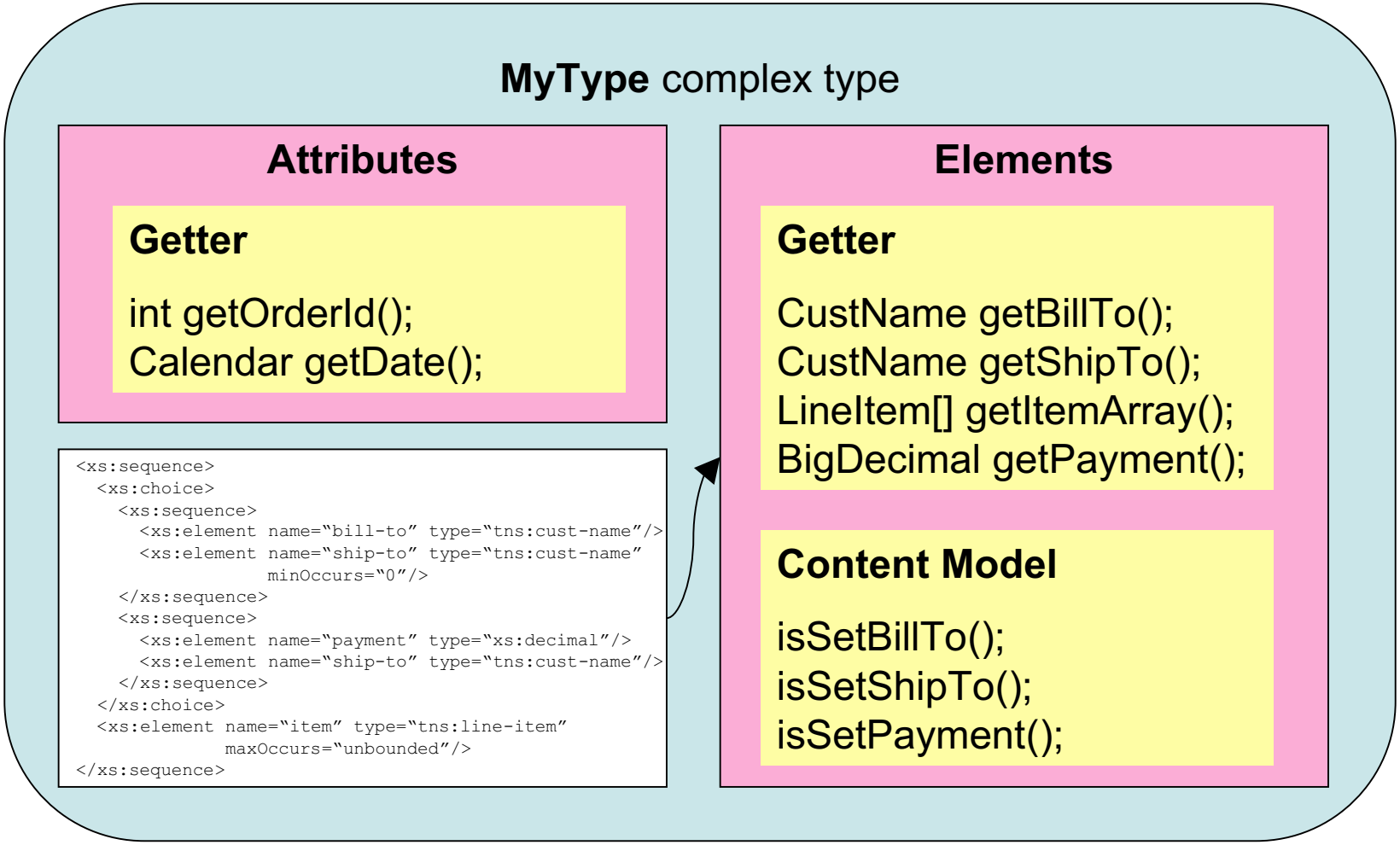
- The answer: *getters for simple type properties* are special.
  - A “simple type getter” is generated that tunnels through to the natural Java value.
  - An “simple type xgetter” provides the XMLBean value.

```
int i = xobj.getQuantity();
XmlInt xi = xobj.xgetQuantity();
```

Content Models in Schema



Content Models in Java



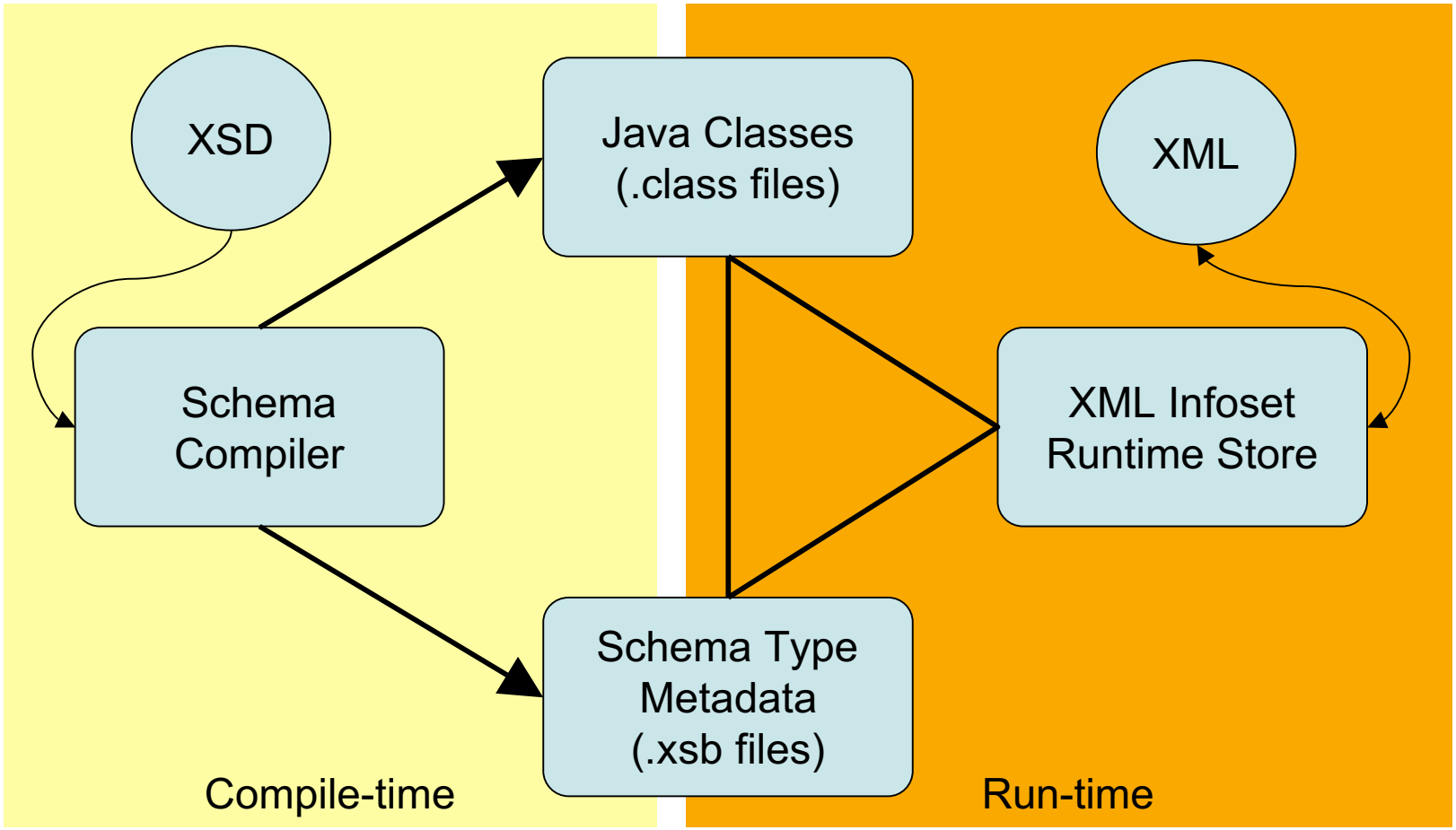
Tricky Content Model Issues

- By-name getters are easy to use:
  - getShipTo() equivalent to ./ship-to
  - getItemArray()[0] equivalent to ./item[1]
- What about when ordering is significant?
  - (buy sell)|(sell buy)
  - getBuy(); getSell(); does not expose order.
- And what about ordering for setters?

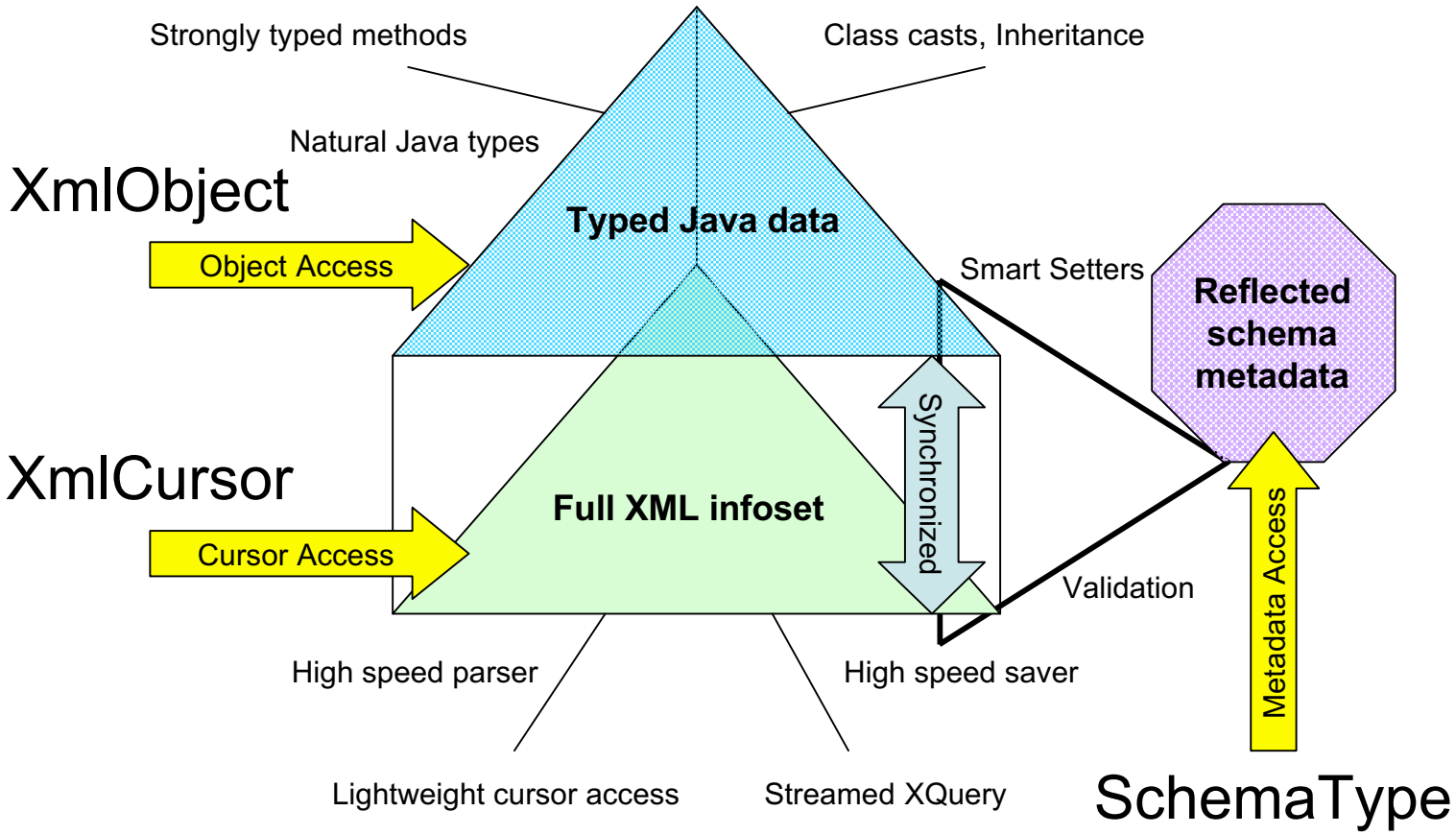
Tricky Content Model Solution

- Answer #1: setters are smart
  - Read the code to see exactly how (☺)
- Answer #2: instances maintain 100% XML infoset, provide tree access when needed.
  - XmlCursor can be used to discover element order or manipulate raw XML tree
  - Also works for access to wildcards, mixed content, and invalid content.

Now, Runtime


XSD
Java Classes
(.class files)
XML
Schema
Compiler
XML Infoset
Runtime Store
Schema Type
Metadata
(.xsb files)
Compile-time
Run-time

Runtime


Strongly typed methods
Class casts, Inheritance
Natural Java types
XmlObject
Typed Java data
Object Access
Smart Setters
Reflected schema metadata
XmlCursor
Full XML infoset
Synchronized
Cursor Access
Validation
Metadata Access
High speed parser
High speed saver
Lightweight cursor access
Streamed XQuery
SchemaType

Runtime API

Major parts of the Runtime API

- Factory is the front door
  `XmlObject.Factory.parse methods`
- XmlObject is the base class for all instances
  `xobj.schemaType(), xobj.newCursor(), xobj.validate() methods`
  `CustomerDocument cdoc = (CustomerDocument)xobj; coercion`
- XmlCursor is used to navigate XML trees
  `xcur.toNextElement(), xcur.toNextToken() navigation`
  `xcur provides full XML Infoset visibility`
- SchemaType is the xsd type system reflection API
  `stype.isSimpleType(), stype.getContentModel() reflection`
  `stype provides full XSD Schema Type inspection`

Cursor Approach to XML

- Advantages of XMLCursor
  - Random access and read/write, just like DOM.
  - By avoiding assigning object identity to each location, implementation is free to be more efficient in avoiding allocation.

Schema Object Model

- Extremely useful for understanding XML schemas in terms of their logical components
- Schemas can be statically compiled or dynamically loaded.
- SOM information is available at runtime, just like Java reflection APIs.

Project Status

- In incubation
- XMLBeans 1.0 complete and usable today
- XMLBeans 2.0 development beginning

This presenation has ended.
Click anywhere to return.

_

